ARM64 Assembly on Apple Silicon — Lab-Based Course

Oct 9, 2026 · @AwareTV

How to use this course

This is a 12-chapter, lab-first course for an Apple Silicon Mac: about one chapter per week at 5–7 hours, roughly 100 hours in total including the capstone. Every chapter ends with a program you write yourself; do not move on until its checkpoint passes.

Each chapter has the same shape. Theory (readings and a video, ~2 h) → Guided lab (build the thing step by step, ~2 h) → Challenge (extend it without hand-holding, ~2 h) → Checkpoint (what must work before you continue). Keep every lab in one git repo: `arm64-course/ch00 … ch11/ capstone/`, each with a `Makefile`, your `.s` files and a `test.sh`.

Prerequisites. You can read C (pointers, arrays, structs, function calls) and use Terminal. Python is useful for writing test harnesses and the capstone's assembler but is not required. If C is rusty, keep Compiler Explorer open while you read Chapters 1–5; seeing C and its assembly side by side is the fastest refresher.

What you need. An M-series Mac on a current macOS, the Xcode Command Line Tools (a Terminal install in Chapter 0; the Xcode app itself is never needed), VS Code with the ARM syntax-highlighting extension and the CodeLLDB debugging extension (also set up in Chapter 0), and git. No virtual machine, no Rosetta, no Linux box: everything runs natively.

Conventions used throughout. AArch64 (A64) syntax as Apple's clang assembler accepts it; lowercase mnemonics; source files end in `.s` (use `.S` when you want the C preprocessor for macros); build with `clang` unless a lab says otherwise; debug with `lldb` (there is no gdb on macOS).

Ch	Topic	Lab deliverable	Hours
0	Toolchain setup	`hello` via raw syscall and via libc; disassemble a C hello	3
1	Architecture & registers	Exit-code calculator stepped through in lldb	5
2	Data processing	Straight-line arithmetic and bit tricks: divmod, align_up, bswap, bit fields, 64-bit constants	6

Ch	Topic	Lab deliverable	Hours
3	Branches, conditions, loops	gcd, ipow, isqrt, popcount, is_prime; branchless min/max/abs	6
4	Memory & addressing	Array and string routines: sum, reverse, strlen, memcpy, insertion sort, binary search	6
5	Functions & the ABI	Assembly library called from C; `printf` called from assembly	8
6	Compiler output & lldb	Predict-then-verify notebook; fix a seeded-bug program	5
7	Strings, I/O, syscalls	`wc`, `cat` and `hexdump` clones	7
8	Floating point & NEON	Dot product and grayscale converter, scalar vs SIMD, benchmarked	8
9	Data structures & interop	Linked list, hash table, `qsort` comparator, tested from C	7
10	Atomics & threads (optional)	Spinlock and atomic counter with pthreads	5
11	Performance (optional)	`memcpy` / `strlen` tuned and benchmarked against libc	6
—	Capstone	Bytecode VM in pure AArch64; JIT as stretch goal	20–30

Chapter 0 — Toolchain setup and your first two programs (3 h)

By the end of this chapter you can assemble, link, run, debug and disassemble an AArch64 program on your Mac, once with raw system calls and once through libc.

Theory (45 min). Read the top of the HelloSilicon README through its Chapter 1 notes; it is the canonical list of what differs between Linux examples and Apple's toolchain (symbol underscores, `svc #0x80`, `@PAGE` / `@PAGEOFF`, reserved `x18`). Skim Apple's Writing ARM64 code for Apple platforms once now; you will re-read it properly in Chapter 5. The course's companion book is Stephen Smith's Programming with 64-Bit ARM Assembly Language (Apress); its examples target Linux, and HelloSilicon ports every one of them to macOS, so you can read the book and the port side by side. The book is optional; every chapter below also lists free readings.

Guided lab.

1. Install Apple's Command Line Tools, which give you clang, the assembler, the linker, lldb and otool without the Xcode app: `xcode-select --install`. Verify with `clang --version`, `lldb --version` and `uname -m` (must print `arm64`).
2. Create `arm64-course/ch00/` under git.
3. Write `hello.s`, the raw-syscall version:

```
.global _main
.align 2                    // Mach-O: 2^2 = 4-byte alignment
_main:
    mov  x0, #1             // fd = stdout
    adrp x1, msg@PAGE       // page address of msg
    add  x1, x1, msg@PAGEOFF  // + offset within the page
    mov  x2, #14            // byte count
    mov  x16, #4            // Darwin syscall number: write
    svc  #0x80
    mov  x0, #0             // exit status
    mov  x16, #1            // Darwin syscall number: exit
    svc  #0x80

.data
msg: .ascii "Hello, world!\n"
```

4. Build it the long way once, so you see the pieces: `as -o hello.o hello.s` then `ld -o hello hello.o -lSystem -syslibroot $(xcrun -sdk macosx --show-sdk-path) -e _main -arch arm64`. Run `./hello`, then `echo $?` to see the exit status.
5. Build it the short way, which you will use from now on: `clang -o hello hello.s`.
6. Write `hello2.s`, the libc version. Two new things appear: saving the frame pointer and link register, and keeping `sp` 16-byte aligned before `bl`.

```
.global _main
.align 2
_main:
    stp  x29, x30, [sp, #-16]!   // push fp and lr, sp stays 16-aligned
    mov  x29, sp
    adrp x0, msg@PAGE
    add  x0, x0, msg@PAGEOFF
    bl   _puts                   // libc: puts(msg)
    mov  w0, #0                  // return 0 from main
    ldp  x29, x30, [sp], #16     // pop fp and lr
    ret

.data
msg: .asciz "Hello from libc"
```

7. Build with `clang -o hello2 hello2.s`. Deliberately remove the `stp` / `ldp` pair, rebuild, run: note the crash, then put it back. That crash is Chapter 5's whole subject.
8. Write `hello.c` with `printf`, then look at what clang emits: `clang -O1 -S hello.c -o hello_c.s` (readable assembly) and `clang -O1 -c hello.c && otool -tv hello.o` (disassembly of the object). Match each line to something in your `hello2.s`.
9. Start a `Makefile` template: `%: %.s` → `clang -g -o $@ $<`. Keep `-g` on always; it is what makes lldb show source lines.
10. First debugger session: `lldb ./hello`, then `b _main`, `run`, `si` (step one instruction), `register read x0 x1 x2 x16`, and `memory read -fc -c14 $x1` after the `add` executes. Quit with `q`.

VS Code setup. VS Code is the only editor the course uses.

1. Install two extensions from the Marketplace: **ARM** (publisher dan-c-underwood) for A64 syntax highlighting, and **CodeLLDB** (publisher vadimcn) for debugging. CodeLLDB bundles its own lldb and runs natively on Apple Silicon.
2. Add `.vscode/tasks.json` with a build task whose command is `clang -g -o ${fileBasenameNoExtension} ${file}` and make it the default build task (⇧⌘B).
3. Add `.vscode/launch.json` with a CodeLLDB configuration: `"type": "lldb"`, `"request": "launch"`, `"program": "${workspaceFolder}/ch00/hello"`. Breakpoints set in `.s` files work, the Variables panel has a Registers group, and the command palette's "Open Disassembly View" lets you step one instruction at a time.
4. Keep a Terminal panel open anyway: `otool`, `make` and lldb's batch mode are quicker from the shell, and every command in this document is written for the shell.

Challenge. Make `hello.s` print its message three times using one loop, with the counter in `x19`. You will have to look up `subs` and `b.ne` yourself, from the A64 guide linked in Chapter 1.

Checkpoint. Both programs run; you can explain every line of `hello.s` aloud, including why `adr` would not have worked for `msg`; `otool -tv` output no longer looks foreign.

Errors you will meet. `symbol(s) not found for architecture arm64: _main` means a missing `.global _main` or a missing underscore. A crash on the line after `adr x1, msg` means the data lives in another section: use `adrp` / `@PAGEOFF`. A crash inside `puts` or `printf` almost always means `sp` was not 16-byte aligned at the call.

Chapter 1 — Architecture fundamentals and the register file (5 h)

AArch64 is a load/store RISC machine: 31 general-purpose 64-bit registers, every instruction exactly 32 bits wide, and only loads and stores touch memory. This chapter's goal is to know the register file and its conventions cold, because every later chapter

assumes it.

Theory (2 h).

- Arm's free Learn the architecture: A64 Instruction Set Architecture guide (PDF): read the first three sections, through the register descriptions. This guide is the course's primary reference; the other Arm guides live on the Learn the Architecture A-profile page.
- Perry Kivolowitz's free online asm_book: the opening chapters on registers and the "hello" program. It was written for students on Macs and Linux boxes alike, uses the C runtime the same way this course does, and is the best free companion text.
- Raymond Chen's 25-part series *The AArch64 processor (aka arm64)*: index of every part. Read parts 1 and 2 now. The series targets Windows, so skip parts 20–22 and 25 (Windows calling convention) and use Apple's ABI document instead; everything else is pure instruction-set material and excellent.
- Video (30 min): how hello world for arm64 assembly really works (apple silicon). Watch it after Chapter 0, not before; it will confirm what you already did and fill in why.

What to memorise.

- `x0` – `x30` are 64-bit; `w0` – `w30` are their low 32 bits. Writing a `w` register zeroes the upper 32 bits.
- `sp` is the stack pointer and must be 16-byte aligned whenever it is used as a base address. `pc` cannot be read directly; `adr` gives you a pc-relative address. `xzr` / `wzr` read as zero and discard writes.
- Roles (AAPCS64 plus Apple): `x0` – `x7` arguments and results; `x8` indirect-result pointer; `x9` – `x15` scratch; `x16` / `x17` intra-procedure scratch (`x16` also carries the Darwin syscall number); `x18` reserved by Apple, never touch it; `x19` – `x28` callee-saved; `x29` frame pointer; `x30` link register.
- The NZCV flags change only on instructions that ask for it: the `s`-suffixed forms (`adds` , `subs` , `ands`) and `cmp` , `cmn` , `tst` .
- Operand order is destination first: `add x0, x1, x2` means `x0 = x1 + x2` . Immediates take a `#` .
- Memory is little-endian; Apple Silicon uses 16 KB pages (this matters for `mmap` in the capstone).

Guided lab: exit-code calculator.

1. Write `calc.s` whose `_main` computes `(a*b + c) - d` from constants placed in `x1` – `x4` with `mov` , using `mul` , `add` and `sub` , and returns the result in `w0` . Exit status is only 8 bits, so pick constants whose result is under 256. Run `./calc; echo $?` .

2. Step it in lldb: `b _main`, `run`, then alternate `si` and `register read x0 x1 x2 x3 x4` until `ret`. Use `p/x $x0` for hex.
3. Prove the zero-extension rule: insert `mov x0, #-1` then `mov w0, #5`, step past both, and read `x0`.
4. Read `sp` at entry with `p/x $sp`; it ends in 0 because it is 16-byte aligned.
5. Write `regs.s` that loads `x19`–`x28` with 19–28 and returns 0. Confirm the values in lldb before `ret`. Nothing crashes, because you called no one; Chapter 5 explains why clobbering these would matter once you do.

Challenge. Return the number of trailing zero bits of a constant as the exit status, using `rbit` then `clz`. Find both instructions in the A64 guide yourself, including why `clz` on `w` and `x` differ.

Checkpoint. You can name every register group's role without notes; `calc` returns the expected value; you can single-step and inspect registers in lldb without consulting Chapter 0.

Chapter 2 — Data processing: arithmetic, logic, shifts, immediates (6 h)

Everything that is not a load, a store or a branch is data processing. The hard part is not the mnemonics; it is knowing which constants can be encoded and how the free shift or extend on the second operand works.

From here on, labs are assembly functions tested from a C harness. The only calling-convention rule you need until Chapter 5 is the leaf-function rule: arguments arrive in `x0`–`x7`, the result goes back in `x0` (or `x0`+`x1` for a 16-byte struct), temporaries live in `x9`–`x15`, and you never touch `x19`–`x29`, `x18` or `sp`. A leaf function is just a global label ending in `ret`:

```
.global _align_up16
.align 2
_align_up16:                // uint64_t align_up16(uint64_t x)
    add  x0, x0, #15
    and  x0, x0, #-16       // -16 = 0xFFFF...FFF0 is a legal logical
immediate
    ret
```

Build with `clang -g -o test test.c bits.s`; the harness `test.c` declares `uint64_t align_up16(uint64_t);` and checks results with `assert`.

Theory (2 h).

- A64 guide: the data-processing sections (arithmetic, logical, shifts, bit manipulation, multiply and divide).
- Raymond Chen parts 4–10: add/sub, multiply/divide, bitwise ops and their strange immediates, bitfields, shifts and rotation, sign and zero extension, loading constants.
- asm_book: the chapters on integer arithmetic, bit operations and `mov` / `movz` / `movk`.
- Compiler Explorer: pick `clang` with `--target=arm64-apple-darwin -O2` and type small C expressions (`x * 9`, `x / 10`, `x % 7`, `(x + 15) & ~15`). Keep it open for the rest of the course; every time you wonder how to do something, ask the compiler first.

What to learn.

- Arithmetic: `add`, `sub`, `adds`, `subs`, `neg`, `cmp`, `cmn`; `mul`, `madd`, `msub`, `umulh`, `smulh`; `udiv`, `sdiv`. There is no remainder instruction: `rem = a - (a / b) * b` is one `udiv` plus one `msub`.
- Logic and bits: `and`, `orr`, `eor`, `bic`, `ands`, `tst`; `lsl`, `lsr`, `asr`, `ror`; `ubfx`, `sbfx`, `bfi`, `bfxil`; `clz`, `rbit`, `rev`, `rev16`.
- Two immediate families. Arithmetic immediates are 12 bits, optionally shifted left by 12. Logical immediates must be a repeating bit pattern, which is why `and x0, x0, #0xFF` assembles and `and x0, x0, #0x1234` does not (load it into a register first).
- Building constants: `mov` handles what one `movz` / `movn` can; longer constants are `movz` + up to three `movk`, or `ldr x0, =0x123456789ABCDEF0`, which the assembler turns into a literal-pool load.
- The second operand can carry a shift (`add x0, x1, x2, lsl #3` computes `x1 + x2*8`) or an extend (`add x0, x1, w2, uxtw`), for free.
- Mixing widths: `sxtw`, `uxtb`, `sxth` and friends, and when writing a `w` register already does the job.

Guided lab: `bits.s`, eight leaf functions.

1. `mad(a, b, c)` returns `a*b + c` in one instruction.
2. `divmod(a, b)` returns quotient in `x0` and remainder in `x1` (declare it in C as returning a two-field struct).
3. `align_up16(x)` as above, then `align_up(x, 64)` where the mask is built from the argument with `neg`.
4. `bswap64(x)` with `rev`; `rotl64(x, n)` with `neg` + `ror`.
5. `field(x)` returning bits 12..19 of `x` with one `ubfx`; `set_field(x, v)` with `bfi`.
6. `big_const()` returning `0xDEADBEEFCAFEBABE` using `movz` + `movk`; then a second version using `ldr =`; compare the two in `otool -tv`.
7. `mul10(x)` and `mul9(x)` using shifted operands only, no `mul`.

8. `ilog2(x)` as `63 - clz(x)`; `byte_sign_extend(b)` with `sxtb` from a `uint8_t` argument.

Write `test.c` with at least three `assert`s per function, including edge values (0, 1, `UINT64_MAX`).

Challenge. Implement `udiv` by 10 without `udiv`, using the multiply-high trick (`umulh` with the magic constant clang emits for `x / 10`; read it off Compiler Explorer and work out why it is correct).

Checkpoint. All asserts pass; you can predict, before assembling, whether a given immediate will be accepted; you can explain the `otool` output of both `big_const` versions.

Chapter 3 — Control flow: branches, conditions and loops (6 h)

A64 has no conditional execution of ordinary instructions; it has conditional branches, compare-and-branch on a register, test-a-bit-and-branch, and a small family of conditional-select instructions that let you avoid branching altogether. Learn all four and when each one is the natural choice.

Theory (2 h).

- A64 guide: the "Program flow" sections (loops, conditional branches, condition codes, `csel` family).
- Raymond Chen parts 15–17: control transfer, conditional execution, manipulating flags.
- asm_book: the chapters on `if`, loops and `switch`.
- Compiler Explorer: compile a `for` loop, a `while` loop, `abs()`, `max()` and a 5-case `switch` at `-O2` and read what clang does. Note how often it reaches for `csel` and `cbz` instead of `cmp` + `b.cond`.

What to learn.

- `b label` and `b.cond label` with the condition codes `eq ne hs lo mi pl vs vc hi ls ge lt gt le`; which ones are for signed and which for unsigned comparisons.
- `cbz`, `cbnz` (branch on register zero/non-zero, no flags needed) and `tbz`, `tbnz` (branch on one bit).
- `cmp`, `cmn`, `tst` and `ccmp` for chained conditions.
- Conditional selects: `csel`, `csinc`, `csinv`, `csneg`, and the aliases `cset`, `csetm`, `cinc`, `cneg`.
- Loop shapes: counted loop with `subs` + `b.ne`, pointer-walk loop with `cbnz`, `do`-`while` versus `while`, and why the compiler puts the test at the bottom.

- Branch targets are ±128 MB for `b`, ±1 MB for `b.cond` / `cbz`, ±32 KB for `tbz`; the assembler will tell you when you exceed them.

Guided lab: `flow.s`, tested from C.

1. `gcd(a, b)` with Euclid's remainder loop (`udiv` + `msub` from Chapter 2).
2. `ipow(base, exp)` by square-and-multiply, one `tbz` on the low bit of `exp` per iteration.
3. `isqrt(n)` by bisection over `[0, 2^32)`.
4. `popcount(x)` as a loop (`and`, `lsr`, `add`), then again as a branchless loop using `x & (x - 1)`.
5. `collatz_len(n)` returning the number of steps to reach 1; use `tbz` to pick even/odd.
6. `is_prime(n)` by trial division up to `isqrt(n)`, calling nothing (inline the bisection or recompute).
7. Branchless `abs64`, `max64`, `min64`, `sign64` (returns -1/0/1) using `cmp` plus `csel` / `cneg` / `cset`; verify with `otool -tv` that no `b.` instruction exists in them.
8. `fizzbuzz_code(n)` returning 0, 1, 2 or 3 (plain, Fizz, Buzz, FizzBuzz) with `ccmp` so there is a single conditional branch.

Challenge. `clamp(x, lo, hi)` in four instructions with no branch; then a `switch` on `n` in 0..7 returning distinct constants, written first as a compare chain and then as a computed branch (`adr` + `add` + `br` into a table of equal-size code blocks).

Checkpoint. All asserts pass; you can state, for each condition code, whether it belongs to a signed or unsigned comparison; you reached for `csel` at least twice without being told to.

Chapter 4 — Memory: loads, stores, addressing modes and data sections (6 h)

Only `ldr` / `str` and their relatives touch memory, so pointer code in A64 is about choosing the right addressing mode: scaled immediate, pre- or post-indexed, register offset with a shift, or pc-relative through `adrp`. This is also where Mach-O's `@PAGE` / `@PAGEOFF` syntax stops being a mystery.

Theory (2 h).

- A64 guide: "Loads and stores" in full (addressing modes, sizes, sign-extending loads, `ldp` / `stp`, pc-relative addressing).
- Raymond Chen parts 3, 11 and 12: addressing modes, loading addresses, memory access and alignment.
- asm_book: arrays, pointers and structs chapters.

- ARM64 assembly STR and LDR instructions for Apple Silicon: a short walkthrough that watches `adrp` / `add` resolve in lldb and reads the bytes back with `memory read`; reproduce it.

What to learn.

- Sizes: `ldr x` / `w` (8/4 bytes), `ldrb`, `ldrh` (zero-extend), `ldrsb`, `ldrsh`, `ldrsw` (sign-extend); matching `str`, `strb`, `strh`.
- Addressing modes: `[x0]`; `[x0, #off]` (unsigned, scaled by access size); `ldur` / `stur` for small negative or unscaled offsets; `[x0, #16]!` pre-index and `[x0], #16` post-index (both write back to `x0`); `[x0, x1]`, `[x0, x1, lsl #3]`, `[x0, w1, uxtw #2]`.
- `ldp` / `stp` move two registers at once; `stp x29, x30, [sp, #-16]!` is the canonical push.
- Address formation: `adr` (±1 MB, same section only) versus `adrp` + `add ... @PAGEOFF` (anywhere in the image). For symbols from other libraries, `adrp x0, _sym@GOTPAGE` then `ldr x0, [x0, _sym@GOTPAGEOFF]`.
- Sections and directives: `.text`, `.data`, `.bss` (or `.comm` / `.lcomm`), `.ascii`, `.asciz`, `.byte`, `.hword`, `.word`, `.quad`, `.space`, `.p2align`. Remember that `.align n` in Mach-O means 2^n bytes.
- Alignment: unaligned loads work for ordinary data on Apple Silicon but `sp` must stay 16-aligned, and `ldp` / `stp` with `sp` as base will fault if it is not.

Guided lab: `mem.s`, tested from C.

1. `sum64(const int64_t *a, size_t n)` with a post-indexed load; `min64` / `max64` over an array.
2. `reverse64(int64_t *a, size_t n)` in place with two pointers.
3. `my_strlen(const char *s)` using `ldrb` + `cbnz`; `my_strcmp`.
4. `my_memcpy(dst, src, n)` byte by byte, then 16 bytes at a time with `ldp` / `stp` and a byte tail.
5. `insertion_sort(int64_t *a, size_t n)` and `bsearch64(const int64_t *a, size_t n, int64_t key)` returning the index or -1.
6. `day_name(unsigned d)` returning a `const char *` from a table of `.quad` pointers in `.data`, loaded with `adrp` / `add` and `ldr x0, [x1, x0, lsl #3]`.
7. A struct lab: given `struct point3 { double x, y, z; int32_t id; }`, write `point_id(const struct point3 *p)` and `point_set_z(struct point3 *p, double z)`; compute the offsets by hand first, then check them against `offsetof` in the C harness.

Challenge. `my_memset(dst, byte, n)` that replicates the byte into a 64-bit pattern with multiply-by-`0x0101010101010101`, stores 16 bytes per iteration with `stp`, and handles the head and tail; benchmark it against libc `memset` on 1 MB with `clock_gettime`.

Checkpoint. All asserts pass; `otool -tv mem.o` shows you chose post-index for the pointer walks; you can explain why `adr` cannot reach `.data` but `adrp` can.

Chapter 5 — Functions, the stack, and the Apple AAPCS64 ABI (8 h)

The ABI is the contract that lets your assembly call libc and lets C call your assembly; get it wrong and the program crashes somewhere far from the bug. On macOS the contract is AAPCS64 plus a handful of Apple changes, the most important being that variadic arguments go on the stack.

Theory (3 h).

- Apple's Writing ARM64 code for Apple platforms, now read completely. Write down every line where Apple says it diverges from Arm's standard.
- Arm's Procedure Call Standard for the Arm 64-bit Architecture (AAPCS64): chapter 6, the base standard (register roles, stack, parameter passing, result return). Skim the rest.
- Raymond Chen parts 21–24: prologues and epilogues, other prologue shapes, common patterns, a full code walkthrough. The prologue instruction sequences are identical on macOS; only the Windows-specific argument rules differ.
- asm_book: the chapters on functions, the stack and recursion.
- HelloSilicon's notes on the chapters that call `printf`: they show the stack layout for variadic calls on Apple.

What to learn.

- `bl` writes the return address to `x30` and `ret` jumps to it; any function that calls another must save `x30` or it will return into itself.
- Caller-saved: `x0`–`x17` (and `v0`–`v7`, `v16`–`v31`); anything you need across a `bl` goes in `x19`–`x28` or on the stack. Callee-saved: `x19`–`x28`, `x29`, `x30`, and the low 64 bits of `v8`–`v15`.
- The canonical prologue and epilogue:

```
_f:
    stp  x29, x30, [sp, #-32]!   // frame record, plus 16 bytes for x19/x20
    mov  x29, sp
    stp  x19, x20, [sp, #16]
    ...
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret
```

- `sp` is 16-byte aligned at every call; Apple requires `x29` to always point at a valid frame record; leaf functions may use the 128-byte red zone below `sp` without moving it.
- Arguments 1–8 in `x0` – `x7`, floating-point in `d0` – `d7`; the 9th and later on the stack at `[sp]` when `bl` executes, which is `[x29, #16]` inside the callee after the standard prologue.
- Apple divergence 1: variadic arguments (`printf`'s `...`) are never in registers. Each goes in an 8-byte stack slot starting at `[sp]`, including doubles.
- Apple divergence 2: `char` is signed on Apple platforms (unsigned on Linux AArch64). Divergence 3: small stack arguments are packed to their natural alignment, not padded to 8 bytes. Divergence 4: `x18` is reserved.
- Results: `x0`, or `x0` + `x1` for 16-byte structs, or written through the pointer passed in `x8` for anything larger.

Calling `printf` from assembly correctly on macOS:

```
    stp  x29, x30, [sp, #-16]!
    mov  x29, sp
    sub  sp, sp, #16             // room for two variadic slots
    adrp x0, fmt@PAGE
    add  x0, x0, fmt@PAGEOFF     // "%ld %ld\n"
    mov  x9, #42
    str  x9, [sp]                // first variadic argument
    mov  x9, #7
    str  x9, [sp, #8]            // second
    bl   _printf
    add  sp, sp, #16
    ldp  x29, x30, [sp], #16
    ret
```

Guided lab: `calls.s` plus `main.c`.

1. `sum_squares(n)`: a non-leaf function that calls your own `square(x)` in a loop, keeping the counter and accumulator in `x19` / `x20` with the prologue above.

2. Recursive `fact(n)` and `fib(n)`; break inside `fib` in lldb at depth 5 and read the chain of frame records with `bt`, `frame select 3`, and `memory read -fx -c8 $sp`.
3. Call libc from assembly: `_strlen` on a literal, `_malloc` 64 bytes, fill it, `_puts` it, `_free` it.
4. `printf` with two integers and a double from assembly, using the stack layout above.
5. Compile `fib.c` with `clang -O1 -S` and match clang's prologue and epilogue to yours line by line; ignore the `.cfi_` directives for now (they describe the frame to the unwinder).
6. Break the rules on purpose, one at a time: clobber `x19` in a function called from a C loop that keeps its counter in a register; misalign `sp` by 8 before `bl _printf`; forget to save `x30` in a non-leaf. Record what each failure looks like, because you will meet all three again in the capstone.
7. Struct results: return a two-`int64_t` struct (comes back in `x0`/`x1`) and a three-`int64_t` struct (written through `x8`). Confirm both against Compiler Explorer output for the same C.

Challenge. A function taking 10 integer arguments that returns the sum of the 9th and 10th, read from the caller's stack; then `printf_ints(fmt, a, b)` in assembly that forwards two integers to `printf` with a correctly built variadic frame.

Checkpoint. You can write the prologue and epilogue from memory; you can say which registers survive a `bl` without looking; `printf` from assembly prints the right numbers; `bt` in lldb makes sense to you frame by frame.

Chapter 6 — Reading compiler output and debugging with lldb (5 h)

After five chapters you can write assembly; this chapter makes you fluent at reading it and at finding bugs in it, which is most of what assembly skill is used for in practice. The method is predict-then-verify: guess what clang will emit, then check.

Theory (1.5 h).

- LLDB tutorial and the GDB to LLDB command map; keep the map open until the lldb commands are in your fingers.
- Raymond Chen part 23 (common code-generation patterns) and part 24 (a complete code walkthrough), read a second time, now with Compiler Explorer beside you.
- Chapter 1 of the Arm Learn the architecture: A64 ISA guide on instruction encoding, so that `otool`'s raw words (`-t` without `-v`) make sense.

Tools to master.

- `clang -O1 -S -fno-asynchronous-unwind-tables file.c` gives readable assembly without `.cfi` noise; `-O0` shows every local on the stack; `-O2` shows what the compiler really thinks.
- `otool -tV a.out` (disassemble, symbolic), `otool -tv -p _sym`, `nm a.out`, `objdump -d --macho a.out`, `size a.out`.
- lldb essentials: `b _sym`, `b file.s:12`, `run`, `si`, `ni`, `finish`, `c`, `register read`, `register read -f x`, `p/x $x0`, `memory read -fx -c8 $sp`, `memory read -s1 -fc -c32 $x0`, `disassemble -p` (around pc), `disassemble -n _sym`, `bt`, `frame select 2`, `watchpoint set expression -- $x0 + 8`, `thread step-inst`, `process status`.
- `lldb --batch -o "b _main" -o run -o "disassemble -p" ./a.out` for scripted sessions; useful in `test.sh`.

Guided lab: the predict-then-verify notebook.

1. Create `predictions.md`. For each of these C functions, write the assembly you expect at `-O2` before compiling: `strlen`, a loop summing an `int32_t` array, `abs`, a 3-way `switch`, a function returning a small struct, a tail-recursive function.
2. Compile each with `clang -O2 -S` and diff against your prediction. Write one line per mismatch about what the compiler did and why (it will use `csel`, `ldp`, post-index, `madd`, tail calls with `b` instead of `bl`, and it will unroll).
3. Reverse direction: take the five disassembly listings in Raymond Chen part 24 and write the C they came from, then compare with his reading.
4. Bug hunt. Seed these bugs into copies of your Chapter 4–5 code, one per copy, then find each with lldb only, no re-reading the source: a `w` register used where an `x` was needed, an `ldr` of a byte with `ldr` instead of `ldrb`, a missing `mov x29, sp`, an off-by-one post-index (`#8` on 4-byte data), a `cmp` with the operands reversed, and a callee-saved register clobbered inside a loop.
5. Learn `.cfi` just enough: compile `fib.c` with and without `-fno-asynchronous-unwind-tables` and note what the directives describe. In your own `.s` files, add `.cfi_startproc` / `.cfi_endproc` with the frame directives clang emits so `bt` is accurate through your functions.

Challenge. Use `otool -t` (no `-v`) on a small function and decode three instructions by hand from the 32-bit words using the encoding tables in the A64 guide; confirm with `otool -tv`.

Checkpoint. Your mismatch rate on the predictions drops below one per function on a second batch of five new functions; you found all six seeded bugs in lldb within 10 minutes each.

Chapter 7 — Strings, I/O and macOS system calls (7 h)

This chapter turns your functions into real command-line programs: reading `argv`, converting numbers to text and back, and doing file I/O both through raw Darwin system calls and through libc. You will finish with three Unix tools written entirely in assembly.

Theory (2 h).

- The Darwin convention on arm64: syscall number in `x16`, arguments in `x0` – `x7`, `svc #0x80`; on failure the carry flag is set and `x0` holds `errno`. Numbers come from XNU's syscalls.master: `exit` 1, `read` 3, `write` 4, `open` 5, `close` 6, `munmap` 73, `mmap` 197, `lseek` 199. Apple treats these numbers as private, so real programs call libc's `_read` / `_write` / `_open`; raw `svc` is for understanding what libc does.
- Flag values differ from Linux: on macOS `O_RDONLY` is 0, `O_WRONLY` 1, `O_CREAT` 0x200, `O_TRUNC` 0x400. Look constants up in the SDK headers (`xcrun --show-sdk-path` then `usr/include/sys/fcntl.h`), never from a Linux tutorial.
- `_main` receives `argc` in `x0` and `argv` in `x1`, exactly like C.
- asm_book: the chapters on strings, `argv` and number formatting.
- HTTP Server in arm64 assembly, Apple Silicon M1: read it for the syscall and socket plumbing; it is the model for this chapter's challenge.
- Stephen Smith's own post Apple M1 assembly language hello world for his account of the Linux-to-macOS differences.

Guided lab: three tools.

1. Conversions in `conv.s`, tested from C: `u64toa(v, buf)` returning the length (repeated `udiv` / `msub` by 10 into a 20-byte buffer, then reverse), `i64toa` with sign, `u64tohex` using a lookup table of `.ascii "0123456789abcdef"`, and `atou64(s)` that stops at the first non-digit.
2. Output helpers in `io.s`: `write_str(fd, s)` (calls your `my_strlen` then `write`), `print_u64(v)`, `print_hex(v)`, each first with a raw `svc`, then again by calling `_write`. Keep both; they will differ by only a few lines.
3. `cat`: loop over `argv[1..]`, `open` each, `read` into a 16 KB `.bss` buffer in a loop until `read` returns 0, `write` each chunk to fd 1, `close`. On any carry-set error print the file name and `errno` to fd 2 and continue. No arguments means copy stdin.
4. `wc`: read stdin to end; count bytes, newlines and words with a two-state machine (in-word/not-in-word) using `cmp`, `ccmp` and `cinc`; print three numbers with `print_u64`.
5. `hexdump`: 16 bytes per line, the offset in 8 hex digits, bytes in hex, then the printable-ASCII column with `.` for non-printables (`cmp` + `csel` picks the character).

Challenge. A minimal HTTP server: `socket`, `bind`, `listen`, `accept`, `read` the request, `write` a fixed `200 OK` response, `close`, loop. Use the libc wrappers rather than raw syscalls for the socket calls, and test with `curl localhost:8080`.

Checkpoint. `./cat a b | diff - <(cat a b)` prints nothing; your `wc` matches `/usr/bin/wc` on three files including an empty one; you can explain where `errno` comes from after a failed syscall and how libc turns the carry flag into `-1`.

Chapter 8 — Floating point and NEON SIMD (8 h)

The 32 vector registers `v0` – `v31` do double duty: `d0` is a scalar double, `s0` a scalar float, and `v0.4s` is the same register seen as four floats processed in one instruction. Apple Silicon has NEON (Advanced SIMD) but no SVE, so NEON is the whole SIMD story here.

Theory (3 h).

- Arm Learn the architecture: Introducing Neon (PDF): the register views and lane arrangements.
- Arm Neon Programmer Guide: Coding for Neon (PDF), the assembly-level guide; start from the original blog Coding for Neon, Part 1: Loads and Stores if you want the shorter version first.
- Arm Neon intrinsics reference: every intrinsic shows the instruction it maps to, so it doubles as an instruction search engine.
- A64 guide: the floating-point and SIMD sections; AAPCS64 for `d0` – `d7` argument passing and the callee-saved `d8` – `d15` (low 64 bits only).
- Compiler Explorer with `-O3 -Rpass=loop-vectorize`: see which of your C loops clang vectorises, and what it emits.

What to learn.

- Scalar FP: `fmov`, `fadd`, `fsub`, `fmul`, `fdiv`, `fsqrt`, `fmadd`, `fmsub`, `fneg`, `fabs`, `fmin`, `fmax`, `fcmp` (sets NZCV; unordered sets V), `fcsel`, `fcvt` (float/double), `scvtf` / `ucvtf` (int to FP), `fcvtzs` / `fcvtzu` (FP to int, truncating), `fcvtns` and friends for other roundings.
- FP constants: `fmov d0, #1.0` works only for a small encodable set; everything else is a `.double` in `.data` loaded with `adrp` / `ldr d0, [x0, lbl@PAGEOFF]`.
- Lane notation: `v0.16b`, `.8b`, `.8h`, `.4h`, `.4s`, `.2s`, `.2d`; element access `v0.s[2]`; `dup` to broadcast, `ins` / `umov` / `mov` to move single lanes.
- Vector loads and stores: `ld1` / `st1` with post-index, and the de-interleaving `ld2` / `ld3` / `ld4` (RGB pixels straight into three registers).

- Lane-wise arithmetic: integer `add` / `sub` / `mul` / `mla`, widening `umull` / `umlal` / `uaddl`, narrowing `xtn` / `shrn` / `uqshrn`, saturating `sqadd`; float `fadd` / `fmul` / `fmla` including by-element `fmla v0.4s, v1.4s, v2.s[0]`.
- Reductions and shuffles: `addv`, `uaddlv`, `faddp`, `smaxv`, `umaxv`; `cnt` (per-byte popcount); `cmeq` / `cmgt` masks, `bsl`; `tbl`, `zip` / `uzp` / `trn`, `ext`.
- Apple's assembler also accepts a non-portable shorthand for some NEON forms; write the standard syntax so your code assembles elsewhere (see Writing portable ARM64 assembly).

Guided lab: `simd.s`, benchmarked against C.

1. Scalar FP: `dot_d(a, b, n)` with `fmadd`, `horner(coeffs, n, x)`, `mean_var(a, n, &mean, &var)`. Test against C to 1e-12.
2. `dot_f_neon(a, b, n)`: four lanes with `fmla v0.4s`, a scalar tail loop, then `faddp` twice to reduce. Compare with the scalar version for accuracy (summation order differs) and speed.
3. `vadd_i32(dst, a, b, n)` with `ld1 {v0.4s}, [x1], #16` and `st1`; then unrolled to two vectors per iteration.
4. Grayscale: the C harness reads a binary PPM (`P6`) and writes a PGM (`P5`); your `rgb_to_gray(rgb, gray, npix)` uses `ld3 {v0.16b, v1.16b, v2.16b}` to split channels, `umull` / `umlal` to form `77R + 150G + 29B` in 16-bit lanes, `shrn #8` to narrow, and `st1` to write 16 pixels per iteration. Open the result in Preview.
5. `popcount_buf(p, n)` with `cnt` + `uaddlv`; compare with Chapter 3's loop on 1 MB.
6. Write 2, 4 and 5 again in C with `<arm_neon.h>` intrinsics; compile at `-O3`, diff the assembly against yours, and benchmark all three versions with `clock_gettime(CLOCK_MONOTONIC, ...)`.

Challenge. `memchr` with `cmeq` + `umaxv` to find the first matching byte 16 at a time; or a 4×4 float matrix multiply using by-element `fmla`, checked against a scalar reference.

Checkpoint. NEON results match scalar within tolerance; `rgb_to_gray` is at least 4× faster than your scalar version; you can read `fmla v0.4s, v1.4s, v2.s[0]` aloud and say exactly what each lane receives.

Chapter 9 — Data structures, pointers, C interop and callbacks (7 h)

Assembly has no types, so every struct field, array stride and pointer-to-pointer is an offset you compute yourself. This chapter makes those offsets second nature and completes the C interop picture: heap allocation, function pointers in both directions, and how structs travel by value.

Theory (2 h).

- AAPCS64 chapter 6 again, this time the parts on composite types: 16-byte-or-smaller structs in registers, homogeneous floating-point aggregates in `d` registers, larger structs copied to memory and passed by pointer.
- Apple's ABI document, the data-type table (sizes and alignments) and the signed-`char` note.
- asm_book: the chapters on structs, linked data structures and calling C library functions.
- Compiler Explorer: compile `struct` accessors, a `qsort` call and a function returning a `struct { double x, y; }` at `-O2`.

What to learn.

- Layout rules: fields sit at their natural alignment, the struct's size is rounded up to its largest alignment, arrays of structs use that size as stride. Compute offsets by hand, then pin them with `.set POINT_Z, 16` and use `[x0, #POINT_Z]` so the code reads like C.
- Heap: `_malloc`, `_calloc`, `_realloc`, `_free`; the pointer comes back in `x0`, keep it in a callee-saved register across further calls.
- Function pointers: `blr x9` calls through a register; a callback you pass to C must obey the full ABI because C will call it like any function.
- C globals from assembly: `adrp x0, _counter@GOTPAGE` then `ldr x0, [x0, _counter@GOTPAGEOFF]` gives the address; your own globals in `.data` / `.bss` use `@PAGE` / `@PAGEOFF`.
- Strings as keys: a hash (FNV-1a is five instructions per byte) plus `my_strcmp` from Chapter 4.

Guided lab: `ds.s`, tested from C and checked with `leaks`.

1. `struct point3 { double x, y, z; int32_t id; }` accessors using `.set` offsets; the C harness asserts your constants against `offsetof` and `sizeof`.
2. Singly linked list of `int64_t`: `list_push(&head, v)` (calls `_malloc`), `list_len`, `list_sum`, `list_reverse` (three-pointer loop), `list_free`.
3. Hash table with chaining: `hash_str` (FNV-1a 64), `ht_insert(ht, key, value)`, `ht_get(ht, key)` over 256 buckets in `.bss`; keys are `strdup`'d C strings.
4. `cmp_i64(const void *, const void *)` in assembly, passed to libc `qsort` both from C and from assembly (`adrp` / `add` the function's address into `x3`, then `bl _qsort`).
5. `for_each(a, n, fn)` that calls a C callback through `blr` for each element; note why `fn`, `a` and `n` must live in `x19`–`x21`.

6. Structs by value: `vec2_add(struct vec2 a, struct vec2 b)` (arrives in `d0`–`d3`, returns in `d0`/`d1`) and `big_sum(struct big b)` where `big` is four `int64_t` (arrives as a pointer in `x0` to a copy the caller made). Confirm both against Compiler Explorer.
7. Binary search tree: `bst_insert`, `bst_find`, `bst_free` (recursive, with `_malloc`'d nodes).

Run every harness under `leaks --atExit -- ./test` and fix anything it reports.

Challenge. A growable array (`vec_push` with `_realloc` doubling, `vec_get`, `vec_len`) used as the backing store for the hash table's buckets instead of linked chains.

Checkpoint. All asserts pass with zero leaks; you can state from memory where a 16-byte struct, a `{double, double}` struct and a 32-byte struct each arrive when passed by value.

Chapter 10 (optional) — Atomics, memory ordering and threads (5 h)

Arm's memory model is weakly ordered: two cores may see each other's stores in different orders unless you say otherwise with acquire/release accesses or barriers. Apple Silicon also implements the LSE atomics (`ldadd`, `cas`, `swp`), so a lock-free counter is one instruction. Skip this chapter on a first pass if the capstone is calling; come back before you write anything multithreaded.

Theory (2 h).

- Raymond Chen part 13 (atomic access) and part 14 (barriers).
- A64 guide: the memory-ordering and barrier sections; the memory-model guides linked from the Learn the Architecture A-profile page for the formal picture.
- Compiler Explorer: compile C11 `atomic_fetch_add`, `atomic_load_explicit(..., memory_order_acquire)` and `atomic_store_explicit(..., memory_order_release)` at `-O2` for `arm64-apple-darwin`, and note which instructions each becomes.

What to learn.

- Exclusive pairs: `ldaxr`/`stlxr` in a retry loop (`cbnz w_status, retry`); `ldxr`/`stxr` without ordering.
- LSE atomics: `ldadd`, `ldadda`/`ldaddl`/`ldaddal` ordering variants, `cas`/`casa`/`casal`, `swp`, `ldset`, `ldclr`.
- Acquire/release: `ldar`, `stlr`, `ldapr`; barriers `dmb ish`, `dsb`, `isb`, and why you almost never want a full `dsb` in user code.
- Rosetta runs x86 code in TSO mode; native arm64 code does not get that, so a data race that "worked" on Intel may fail on Apple Silicon.

- Calling `_pthread_create` from assembly: the thread function is an ordinary ABI-conforming function receiving its argument in `x0`.

Guided lab: `sync.s` with a C harness.

1. Four pthreads each add 1 to a shared counter 1,000,000 times with plain `ldr` / `add` / `str`; print the (wrong) total. Fix it three ways and time each: `ldaxr` / `stlxr` loop, `ldadd`, and `ldaddal`.
2. A spinlock: `lock` with `casa` (or `ldaxr` / `stlxr`) and `yield` in the spin, `unlock` with `stlr wzr`. Use it to protect the Chapter 9 linked list under 4 threads pushing concurrently; `list_len` must equal the total pushes.
3. Message passing: thread A writes a payload then sets a flag with `stlr`; thread B spins with `ldar` then reads the payload. Then replace `stlr` / `ldar` with plain `str` / `ldr` and explain, with the memory model, why the program is now wrong even if it appears to pass.
4. Create and join a thread entirely from assembly, passing a struct pointer as the argument and returning a value through `_pthread_join`.

Challenge. A single-producer single-consumer ring buffer with head and tail indices published through acquire/release only, no locks; verify with a checksum over 100 million items.

Checkpoint. The counter totals are exact under all three fixes; you can say which of `ldadd`, `ldadda`, `ldaddl` and `ldaddal` a mutex unlock needs and why.

Chapter 11 (optional) — Performance and the Apple Silicon microarchitecture (6 h)

Apple's cores are very wide (eight-wide decode, deep out-of-order windows, 128-byte cache lines), so the old intuition "fewer instructions is faster" is often wrong; what matters is dependency chains, branch predictability and memory traffic. This chapter gives you a measurement habit and a few mental models that transfer to any modern core.

Theory (2 h).

- Dougall Johnson's Apple M1 microarchitecture research: the Firestorm overview, then the base instruction tables with per-instruction latency and throughput. Newer M-series cores differ in detail but not in shape.
- Raymond Chen part 23 (common patterns) once more, this time asking of each pattern "what is the dependency chain?".

- Tooling: `clock_gettime(CLOCK_MONOTONIC_RAW, ...)` for in-process timing, `hyperfine` (install via Homebrew) for whole-program timing, Instruments' Time Profiler (`xcrun xctrace record --template 'Time Profiler' --launch ./a.out`) for hot spots; Instruments ships only with the full Xcode app, so treat it as optional. With the Command Line Tools alone, hyperfine plus clock_gettime cover everything in this chapter. There is no `perf` on macOS.

What to learn.

- Latency versus throughput: a chain of dependent `fadd`s runs at latency; independent ones run several per cycle. Multiple accumulators break the chain.
- Branch misprediction costs tens of cycles; `csel` costs one. Data-dependent branches on random data are the usual culprit.
- Memory: sequential access is nearly free, strided access past the line size is not; 128-byte lines on Apple Silicon; the hardware prefetcher handles regular strides.
- Unrolling helps only until the loop body saturates the issue width; measure rather than guess.
- Alignment of code rarely matters here; alignment of data for `ldp q0, q1` and NEON loads sometimes does.

Guided lab: `perf/` benchmarks.

1. A C micro-benchmark harness that calls an assembly function N times on a buffer and prints ns/call and GB/s, with a warm-up and the median of 5 runs.
2. `strlen` three ways: Chapter 4's byte loop, an 8-byte SWAR version (the has-zero-byte trick), and Chapter 8's NEON `cmeq`/`umaxv` version; compare with libc `strlen` for lengths 7, 64 and 4096.
3. `memcpy` three ways: byte loop, `ldp`/`stp` 16 bytes, and `ldp q0, q1`/`stp q0, q1` 32 bytes unrolled twice; compare with libc at 64 B, 4 KB and 1 MB.
4. Branch versus `csel`: count elements above a threshold in a 10-million-element array, once with `b.le` and once with `cinc`, on sorted and then on shuffled data. Explain the four timings.
5. Dependency chains: sum a double array with one accumulator, then two, four and eight; find where it stops improving and compare with the `fadd` latency and throughput in Dougall's tables.
6. Cache strides: walk a 64 MB array with strides of 8, 64, 128, 1024 and 16384 bytes and tabulate ns per access.

Challenge. Bring your `memcpy` within 20% of libc for every size in the lab, and write down which change bought the most.

Checkpoint. You have a benchmark table for every lab item; you can explain each result using latency, throughput, prediction or cache lines rather than instruction count.

Capstone — a bytecode virtual machine in pure AArch64 (20–30 h)

Build a stack-based virtual machine, written entirely in assembly, that loads a bytecode file, runs it, and does I/O; then make it fast, and as a stretch goal make it a JIT. It exercises every chapter: jump-table dispatch (3, 4), a register-resident VM state (1, 5), file loading and number printing (7), heap and data structures (9), benchmarking (11), and, for the JIT, Apple's W^X rules.

The instruction set (write `spec.md` first). One-byte opcodes; `PUSH` carries a 64-bit immediate; branches and `CALL` carry a signed 32-bit offset. Suggested set: `PUSH imm64`, `POP`, `DUP`, `SWAP`, `OVER`; `ADD SUB MUL DIV MOD NEG`; `EQ LT GT`; `AND OR NOT`; `LOAD` (address popped, value pushed) and `STORE` into a 64 KB data area; `JMP off32`, `JZ off32`, `CALL off32`, `RET`; `PRINT` (signed integer plus newline), `EMIT` (one byte), `READ` (one integer from stdin); `HALT`. Define the file format: a 4-byte magic, a code length, then code. Thirty opcodes is plenty; resist adding more until the tests pass.

Build order.

1. `asm.py`: a Python assembler with labels, `.data` reservations and an `.svm` output. Python is the right tool here; the VM is the point, not the assembler.
2. `vm.s`, part 1: `_main` opens `argv[1]`, reads it into an `mmap`-ed or `.bss` buffer, validates the magic, and sets up the VM state in callee-saved registers: `x19` VM stack pointer, `x20` VM program counter, `x21` code base, `x22` data base, `x23` call-stack pointer. Keep them there for the whole run; they are the reason the ABI chapter mattered.
3. `vm.s`, part 2: the dispatch loop. `ldrb` the opcode, bounds-check it, `adrp`/`add` the handler table, `ldr x9, [x10, x9, lsl #3]`, `br x9`. Every handler ends with `b dispatch`. Start with `PUSH`, `ADD`, `PRINT`, `HALT` and run a two-line program before writing anything else.
4. Errors: stack underflow and overflow, division by zero, bad opcode, bad jump target. Each prints a message and the offending `pc` to fd 2 and exits with a distinct code. Test every one deliberately.
5. `tests/`: at least ten programs with expected output, run by `test.sh`: arithmetic; factorial with a loop; recursive `fib(20)` with `CALL`/`RET`; FizzBuzz 1..100 using `EMIT`; bubble sort of 50 values in the data area with `LOAD`/`STORE`; primes below 1000; a program that reads two integers and prints their GCD; three deliberate-error programs.

6. `bench/`: a C implementation of the same VM (a `switch` in a loop, `-O2`) and a timing script. Run `fib(30)` on both. Target: your assembly VM within 10% of C, then faster.
7. Make it faster: convert to threaded dispatch, where each handler loads and branches to the next handler itself instead of jumping back to a central loop; then pre-decode the program into an array of handler addresses so dispatch is `ldr` + `br` only. Re-run the benchmark after each change and keep the table in the README.
8. `README.md`: design, the benchmark table, and three bugs with how lldb found them.

Stretch goal: a template JIT. Translate each bytecode into a fixed native code template (a `PUSH` becomes `movz` / `movk` into a register plus a store through `x19`), patch branch offsets in a second pass, and run the result. On Apple Silicon this requires `mmap` with `MAP_JIT`, `pthread_jit_write_protect_np(0)` while writing and `(1)` before executing, and `sys_icache_invalidate` after writing; a command-line tool without the Hardened Runtime needs no entitlement. Read Apple's Porting just-in-time compilers to Apple silicon and the worked example in JIT in C: injecting machine code at runtime, which shows the exact Apple-only calls. Add the JIT's `fib(30)` time to the benchmark table.

Alternative capstones, if you would rather. (a) A NEON image-processing CLI (box blur, Sobel edges, histogram) for PPM files, benchmarked against C with intrinsics. (b) The Chapter 7 HTTP server extended to serve files from a directory with one pthread per connection, combining Chapters 7, 9 and 10. (c) A `tiny_libc.s` (`strlen`, `memcpy`, `memset`, `strcmp`, `itoa`, a `printf` subset) plus the three Chapter 7 tools rebuilt on it, with a benchmark against libSystem.

Done when: every test passes; the three error paths exit with the right codes; the benchmark table has at least the assembly VM, the C VM and one dispatch optimisation; the README is written. Push the repo and tag it `v1.0`.

macOS-specific gotchas quick reference

Most tutorials, books and videos target Linux or a Raspberry Pi. Everything in them transfers to your Mac except the rows below; check this table whenever a borrowed example fails to assemble, link or run.

Topic	Linux / book habit	macOS arm64 reality
Symbol names	`main`, `printf`	`_main`, `_printf` — every C-visible symbol gets a leading underscore

Topic	Linux / book habit	macOS arm64 reality
Entry point	`_start`, raw `ld`	`_main` via `clang file.s`; raw `ld` needs `-lSystem -syslibroot $(xcrun -sdk macosx --show-sdk-path) -e _main -arch arm64`
Syscall number and trap	`x8`, `svc #0`	`x16`, `svc #0x80`; numbers differ (`write` is 64 on Linux, 4 on Darwin) and Apple calls them private
Syscall error	negative errno in `x0`	carry flag set, errno in `x0`; libc wrappers turn it into `-1` + `errno`
Address of a label	`ldr x0, =label` or `adrp` + `:lo12:`	`adrp x0, label@PAGE` + `add x0, x0, label@PAGEOFF`; imported data via `@GOTPAGE` / `@GOTPAGEOFF`
`.align`	bytes on some targets	power of two: `.align 2` is 4 bytes; write `.p2align 2` to be unambiguous
Variadic calls (`printf`)	extra args in registers	every variadic argument goes on the stack in an 8-byte slot, starting at `[sp]`
`char`	unsigned on Linux AArch64	signed
`x18`	free to use	reserved by Apple; never read or write it
Stack alignment	required by ABI	required and enforced: a misaligned `sp` used as a base faults immediately
Debugger	`gdb`	`lldb` (see the GDB-to-LLDB map)
Disassembler	`objdump -d`	`otool -tv` or `objdump -d --macho`
ELF directives	`.type`, `.size`, `.section .rodata`	not supported; delete them, use `.text` / `.data` or `.section __TEXT,__const`
Static linking	`-static` works	not supported; every binary links libSystem dynamically
Page size	4 KB	16 KB for `mmap` and `mprotect`; `adrp` still works in 4 KB units
Writable + executable memory	`mprotect(RWX)`	W^X: `mmap(..., MAP_JIT)`, `pthread_jit_write_protect_np`, `sys_icache_invalidate`

Topic	Linux / book habit	macOS arm64 reality
NEON syntax	standard Arm mnemonics	Apple's assembler also accepts a shorthand; write the standard form for portability
Memory ordering of translated x86	n/a	Rosetta runs x86 in TSO mode; native arm64 code is weakly ordered

Master link list

Everything the chapters cite, in one place. Free unless marked paid. Video coverage of AArch64 on Apple Silicon specifically is thin; most video courses teach 32-bit ARM or Linux AArch64, which is fine for concepts as long as you translate with the gotchas table above.

Resource	Type	What it is	Chapters
asm_book	Free book	Perry Kivolowitz's AArch64 text, written for Mac and Linux students, C-runtime based	1–9
HelloSilicon	Repo	Every example from the Apress book ported to Apple Silicon, with the differences explained	0, 5, 7
Programming with 64-Bit ARM Assembly Language	Paid book	Stephen Smith, Apress; Linux examples, read alongside HelloSilicon; code repo	0–9
Arm Programmer's Guide for ARMv8-A	Free book	Arm's own introduction to the architecture, A64, NEON and the memory model	1, 8, 10
Assembly on Apple Silicon Mac ARM64	Article series	A systematic macOS-only AArch64 introduction with XNU and clang versions pinned	0–7

Resource	Type	What it is	Chapters
Writing ARM64 code for Apple platforms	Official doc	Apple's ABI divergences: `x18`, variadics, stack, data types	0, 5, 9
Porting just-in-time compilers to Apple silicon	Official doc	`MAP_JIT`, write-protect toggling, cache invalidation	Capstone
Learn the Architecture, A-profile	Official hub	Index of Arm's free guides (ISA, Neon, memory model, exercises)	all
Learn the architecture: A64 ISA	Official guide (PDF)	The course's primary instruction-set reference	1–8
Getting started with A64 exercises	Official guide (PDF)	Arm's own short exercise set; extra practice for Chapters 2–4	2–4
Assembly language basics	Official doc	Arm's gentlest introduction to reading A64	1
Introducing Neon	Official guide (PDF)	Register views, lanes, data types	8
Coding for Neon	Official guide (PDF)	Assembly-level NEON: loads/stores, shifts, permutes, matrix multiply	8
Neon intrinsics reference	Official reference	Searchable; each intrinsic shows its instruction	8, 11
AAPCS64	Official spec	Arm's procedure call standard	5, 9
XNU syscalls.master	Source	Darwin syscall numbers (any recent XNU version has the same low numbers)	7

Resource	Type	What it is	Chapters
Raymond Chen, The AArch64 processor	Article series	25 short parts; instruction-set parts are universal, skip the Windows ABI parts	1–6, 10, 11
Writing portable ARM64 assembly	Article	Mach-O versus ELF differences and Apple's NEON shorthand	0, 8
How to set up and program ARM-64 assembly on Apple Silicon	Article	Second opinion on the Chapter 0 setup	0
STR and LDR instructions for Apple Silicon	Article	Loads, stores and `@PAGE` watched in lldb	4
Apple M1 assembly language hello world	Article	The book's author on the Linux-to-macOS differences	0, 7
HTTP server in arm64 assembly on Apple Silicon	Article	Sockets and syscalls from assembly	7
JIT in C: injecting machine code at runtime	Article	Worked Apple Silicon JIT example with the exact calls	Capstone
Apple M1 microarchitecture research	Reference	Dougall Johnson's latency and throughput tables	11
how hello world for arm64 assembly really works (apple silicon)	Video, 30 min	The one Apple-Silicon-specific getting-started video worth your time	1
Assembly Language Programming with ARM, full tutorial	Video, 2.5 h (freeCodeCamp)	32-bit ARM in QEMU; good for concepts in Chapters 2–5, translate the syntax	2–5
Port your Mac app to Apple silicon (WWDC20)	Video	Context on arm64 on the Mac, Rosetta and endianness; not an assembly tutorial	0

Resource	Type	What it is	Chapters
ARM Assembly 4 Dummies	Paid video course	Low Level's course; Linux-based, with a shell project	optional
Compiler Explorer	Tool	See what clang emits for any C snippet; use `--target=arm64-apple-darwin`	all
LLDB tutorial and GDB to LLDB map	Tool docs	The debugger you will live in	0, 6

Ships with the Xcode Command Line Tools, no download needed: `clang`, `as`, `ld`, `lldb`, `otool`, `objdump`, `nm`, `leaks`, `xctrace`. Install `hyperfine` with Homebrew for Chapter 11.